Code:

```
#include <iostream>
#include <GL/gl.h>
#include <GL/glut.h>

using namespace std;

// Draw 2D square
void drawSquare() {
    glBegin(GL_QUADS);
    glVertex2f(-0.2f, -0.2f);
    glVertex2f(0.2f, -0.2f);
    glVertex2f(0.2f, 0.2f);
    glVertex2f(-0.2f, 0.2f);
    glEnd();


}


void drawTriangle() {
    glBegin(GL_TRIANGLES);
    glVertex2f( 0.0f,  0.2f);   // top
    glVertex2f(-0.2f, -0.2f);   // bottom-left
    glVertex2f( 0.2f, -0.2f);   // bottom-right
    glEnd();

}
```

```
void drawParallelogram() {
    // Offset the top two vertices to the right to create the slant
    glBegin(GL_QUADS);
    glVertex2f(-0.2f, -0.15f);   // bottom-left
    glVertex2f( 0.2f, -0.15f);   // bottom-right
    glVertex2f( 0.3f,  0.15f);   // top-right
    glVertex2f(-0.1f,  0.15f);   // top-left
    glEnd();
}




// Draw text
void drawText(const char* text, float r, float g, float b) {
    glColor3f(r, g, b);
    glRasterPos2f(-0.9f, -0.9f);
    while (*text) {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, *text);
        text++;
    }
}


//window-1:Translation - square
void translate2D() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Original
```

```
    glColor3f(1,1,1);
    drawSquare();

    // Translated
    glPushMatrix();
    glTranslatef(0.7f, 0.4f, 0.5f);
    //glColor3f(1,0.5,0);
    glColor3f(0,1,1);
    drawSquare();
    glPopMatrix();

    drawText("Translation 2D",1,0.5,0);
    glFlush();
}



// window -2 :Scaling - square
void scale2D() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1,1,1);
    drawSquare();

    glPushMatrix();
    glTranslatef(0.5f, 0.0f, 0);
    //glScalef(0.5f, 0.5f, 1);
     glScalef(0.5f, 0.5f, 1);
    glColor3f(0,1,1);
    drawSquare();
```

```
    glPopMatrix();

    drawText("Scaling 2D",0,0,1);
    glFlush();
}


// Window 3: Translation – Triangle
void translateTriangle() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Original triangle (white)
    glColor3f(1, 1, 1);
    drawTriangle();

    // Translated triangle (cyan)
    glPushMatrix();
    glTranslatef(0.5f, 0.0f, 0.0f);
    glColor3f(0, 1, 1);
    drawTriangle();
    glPopMatrix();

    drawText("Translation 2D – Triangle", 0, 1, 1);
    glFlush();
}

// Window 4: Scaling – Triangle
void scaleTriangle() {
    glClear(GL_COLOR_BUFFER_BIT);
```

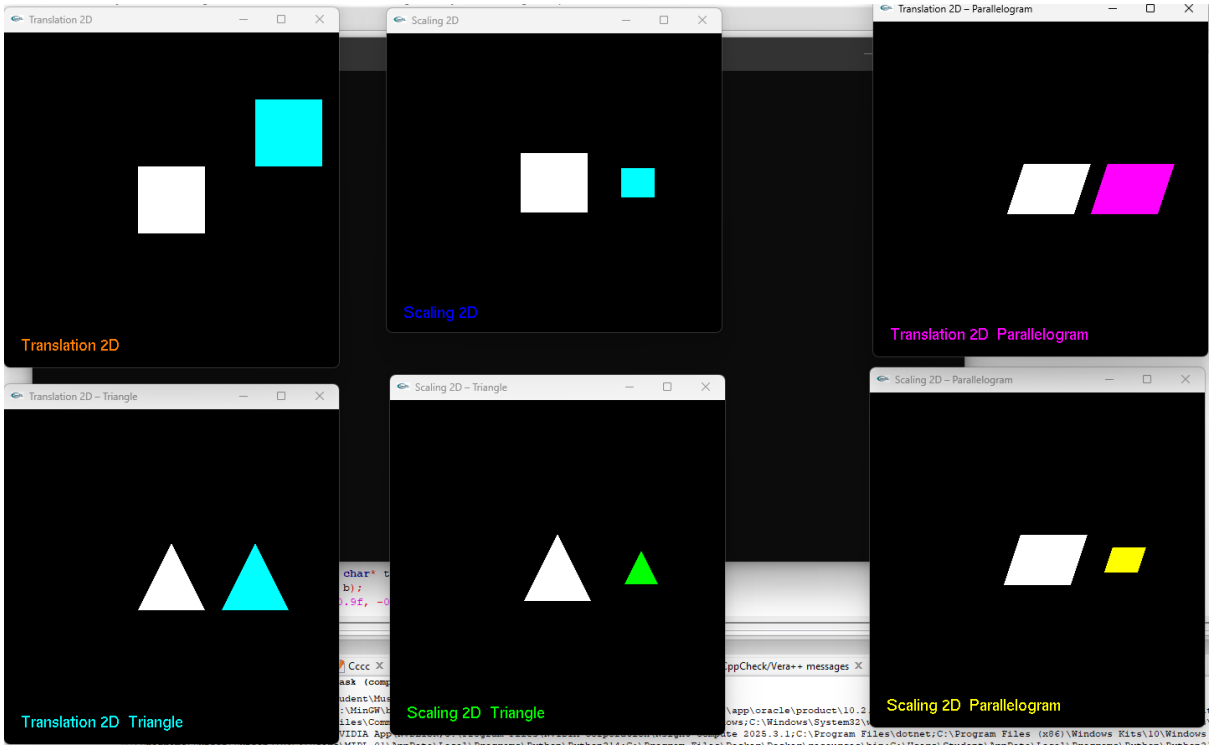
```
    // Original triangle (white)
    glColor3f(1, 1, 1);
    drawTriangle();

    // Scaled triangle (green), moved right
    glPushMatrix();
    glTranslatef(0.5f, 0.0f, 0);
    glScalef(0.5f, 0.5f, 1);
    glColor3f(0, 1, 0);
    drawTriangle();
    glPopMatrix();

    drawText("Scaling 2D – Triangle", 0, 1, 0);
    glFlush();
}

// Window 5: Translation – Parallelogram
void translateParallelogram() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Original parallelogram (white)
    glColor3f(1, 1, 1);
    drawParallelogram();

    // Translated parallelogram (magenta)
    glPushMatrix();
    glTranslatef(0.5f, 0.0f, 0.0f);
    glColor3f(1, 0, 1);
```

```
    drawParallelogram();
    glPopMatrix();

    drawText("Translation 2D – Parallelogram", 1, 0, 1);
    glFlush();
}

// Window 6: Scaling – Parallelogram
void scaleParallelogram() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Original parallelogram (white)
    glColor3f(1, 1, 1);
    drawParallelogram();

    // Scaled parallelogram (yellow), moved right
    glPushMatrix();
    glTranslatef(0.5f, 0.0f, 0);
    glScalef(0.5f, 0.5f, 1);
    glColor3f(1, 1, 0);
    drawParallelogram();
    glPopMatrix();

    drawText("Scaling 2D – Parallelogram", 1, 1, 0);
    glFlush();
}

void setup2D() {
    glClearColor(0,0,0,0);
```

```
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1,1,-1,1); // 2D projection
}


int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(400,400);
      // --- Square windows ---
    glutCreateWindow("Translation 2D");
    setup2D();
    glutDisplayFunc(translate2D);

    glutCreateWindow("Scaling 2D");
    setup2D();
    glutDisplayFunc(scale2D);



    // --- Triangle windows ---
    glutInitWindowPosition(0, 450);
    glutCreateWindow("Translation 2D – Triangle");
    setup2D();
    glutDisplayFunc(translateTriangle);

    glutInitWindowPosition(420, 450);
    glutCreateWindow("Scaling 2D – Triangle");
    setup2D();
```

```
    glutDisplayFunc(scaleTriangle);

    // --- Parallelogram windows ---
    glutInitWindowPosition(840, 0);
    glutCreateWindow("Translation 2D – Parallelogram");
    setup2D();
    glutDisplayFunc(translateParallelogram);

    glutInitWindowPosition(840, 450);
    glutCreateWindow("Scaling 2D – Parallelogram");
    setup2D();
    glutDisplayFunc(scaleParallelogram);



    glutMainLoop();
    return 0;
}
```

Output:


Translation 2D
Translation 2D
Scaling 2D
Scaling 2D
Translation 2D – Parallelogram
Translation 2D Parallelogram
Translation 2D – Triangle
Translation 2D Triangle
Scaling 2D – Triangle
Scaling 2D Triangle
Scaling 2D – Parallelogram
Scaling 2D Parallelogram